CRISPR Efficiency Challenge

Gene Expression Modelling

Objective

Your goal is to build a model that predicts the probability that CRISPR-CAS9 will induce a mutation at specific target sites. The predictor takes as input the location of a target site in the human genome, is trained on CRISPR efficiency measures, and is validated on a separate, unseen subset of training data. This assignment is to be submitted by groups of 1-2 students. In addition to 1) generating data and features, 2) building the model, and 3) writing a lab report, your team will 4) deliver a 10-minute-long presentation to the class discussing your approach and its results.

While we provide suggestions on possible data sources and features, it will be up to you to find and process data, generate required and novel features, and develop an optimal model.

Please keep in mind that we will check for plagiarism or AI-generated code. While *ChatGPT* and other generative models can be useful tools, do not copy and paste mindlessly.

This project will be worth 10% of the class.

Provided Data

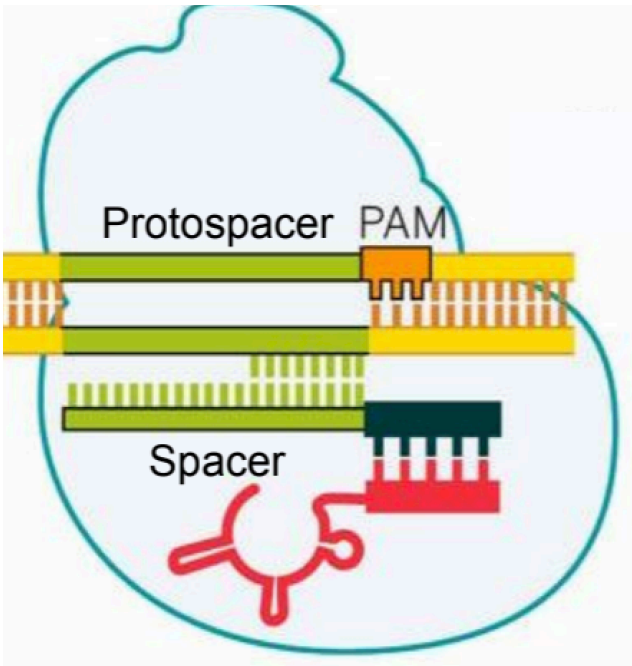
training_data.csv — a table containing specific target sites in the reference human genome that you will use for training the model. The data comes from *HEK293* cells (human embryonic kidney cell line). The data represents off-target positions that CRISPR cleaves, and thus, the PAMs you will find are not always following the NGG pattern. This file will contain the following columns:

- **chr**: the chromosome of the target site
- **strand**: the strand on which the target site is encoded

	• **site_start:** the position at which the target sequence begins • **site_start_hg19:** the position at which the target sequence begins in hg19 build indices (relevant for generating Hi-C related features) • **eff**: a value between 0 and 1 describing the efficiency of CRISPR to induce a mutation at that target site. • **PAM:** the respective PAM that belongs to the sequence; for you to use as a sanity check to make sure you are generating sequence drawing data from fasta files.
testing_data.csv	a table of the specific target sites in the reference human genome, with the same columns as the training dataset but without the 'eff' feature. These are the final prediction targets that will be used to grade the quality of your predictor. Once your model is trained (using crispr_training.csv), you can perform inference on this dataset.
3d_feature.csv	pairwise distances based on Hi-C in HEK293 cells. We provide this data since it is very difficult to work with and clean. Bins are in 10kb resolution, meaning that one bin refers to 10,000 nucleotide segments. The Hi-C data is based on the hg19 genome. The values obtained from this file will represent the spatial density of the chromosomal location you pull.
s_values_human.csv	a file containing the chemical affinity values for codon-anticodon pairing in tRNAs; this is important for calculating tAI weights.

Model Creation

1. Obtain reference GRCh38 chromosome reference genome files as shown in class.
2. For each site in the training dataset, extract the target DNA sequence by pulling the corresponding 20 nucleotides from the respective genome file (you have already created functions that can do this in your homework, modify as needed). As a sanity check, remember that you can look just downstream of the target sequence to check if there is a PAM (it could resemble an NGG pattern, but it's off target, so it's not guaranteed).



3. Obtain the gRNA by creating the reverse complement of the target site you have found in the preceding step.
4. Generate features for each site, target sequence, and gRNA (a list of suggested features is available). This is not an exhaustive list. You may add features as you see fit. In fact, creativity is encouraged. A good direction may be to generate features related to the space around the target sequence and not necessarily including the target sequence (around the sequence that is complementary to the gRNA). A simple example may be generating one-hot encoded nucleotide features of the target sequence to the 5-nt region around the target sequence.

5. Using the generated features, train a model (e.g., using linear regression/stepwise regression/LASSO/ridge/elastic net) that predicts the efficiency supplied in the training dataset. You can test different models as well (e.g., tree-based models).
6. Upon training your final model, you will pass the test dataset through your processing and feature generation pipeline.
7. Once your test set features are available, you can perform inference using the trained regression model to find the test performance.
8. Finally, write a lab report discussing the model you have generated, the features you found most important, the model's performance, and any other relevant material. Additionally, build a 10-minute presentation highlighting the most important information. You will deliver all your code, final predictions, and research paper together.

Challenge Submission

1. Attach a document explaining your predictor:
    - Which model have you chosen? Why?
    - A flowchart of the model
    - Have you added features of your own? If so – what are they (and why have you chosen them)? Which features were selected? Why do you think these features were selected and not others?
2. Attach all needed files to run the model (code and data, if they exist and are not too large).
3. Attach the predicted test set repressions, as calculated in Step 7. The file should come in the same format as crispr_training.csv and should include all the samples found in crispr_testing.csv, with the columns 'chr', 'strand', 'site_start', 'site_end', and 'eff'; however the 'eff' in this file will contain the predicted values that your model outputs.

Challenge Evaluation

The performance of the model you create will be evaluated by measuring the Spearman correlation between the predicted efficiencies that you obtain for the testing set and the true efficiencies measured for the testing set.

Mandatory* and Suggested **Features**

- **Thermodynamics***
  - folding or free energy of a window around the gRNA (I provide you with one such feature)
  - melting temperature of gRNA or target site using the `oligoprop` built-in in MATLAB
- **Nucleotide Composition*:**
  - nucleotides at each position in the target sequence (motif/consensus sequence)
  - total nucleotide counts
  - total dinucleotide counts
  - total trinucleotide counts
- **Gene Distance***: distance of the site to the nearest gene
- **RCBS***: a codon usage bias (hint: https://academic.oup.com/dnaresearch/article-lookup/doi/10.1093/dnares/dsn029)
- **nTE***: normalized translational efficiency is a measure that describes supply and demand in the context of protein translation by considering the tRNA pool as well as the codons that are to be translated (hint: http://gtrnadb.ucsc.edu)
- **Gene expression***: obtain measures of expression for genes near the sites; we leave you to find this data on your own (there are many sources).
- **Chimera scores*:** chimera score of corresponding/adjacent genes* (you should have built an algorithm that calculates this homework)
- Other CUBs: codon usage biases we have discussed (besides nTE and RCBS) of corresponding/adjacent genes
- Adjacent Regions: Features related to the sequence around the target sequence; this could be nucleotide counts, for example

- Hi-C Data: 3D chromosomal conformation features (data is provided)
- Motifs: Sequence motifs of any kind or length

Sample Feature Calculation

1. Dinucleotide Counts (this feature will be provided):
    a. Obtaining the target sequence of 20 nucleotides
    b. Given a sequence *seq* of length *L*, for *i* positions 1 through *L*-1:
        i. Obtain the dinucleotide corresponding to that position (*seq[i:i+1])*
        ii. Increment the count of the corresponding dinucleotide in the given sequence
2. Folding Energy:
    a. Obtain the target sequence of 20 nucleotides
    b. Transcribe the DNA sequence into RNA
    c. Obtain the reverse complement (which will be the gRNA complementary to the target site)
    d. Using the *rnafold* function, obtain the mfe of the whole 20 nucleotide segment (windowing would not be more effective for an already short sequence)

Sample Feature Generation

- chr: 3
- strand: +
- site_start: 49099991
- eff: 0.625

Target Site: `ACTTGACTCGATAGGCTACA`

For each position in the sequence, find the dinucleotide and track its occurrence.

Dinucleotide **TG**: `ACT**TG**ACTCGATAGGCTACA` `count: 1`
Dinucleotide **TA**: `ACTTGACTCGA**TA**GGC**TA**CA` `count: 2`
Dinucleotide **CT**: `A**CT**TGA**CT**CGATAGG**CT**ACA` `count: 3`

Develop the gRNA using the reverse complement.

gRNA Sequence: `UGUAGCCUAUCGAGUCAAGU`

Calculate the folding energy using *rnafold* in MATLAB.

```
grna_seq = 'UGUAGCCUAUCGAGUCAAGU'
energy, structure = rnafold(grna_seq)
```

Continue developing all your other features in a similar fashion.